

Graduate Deep Reinforcement Learning Paper Chains

Comprehensive Project Report

Your Name

Partner Name

Course: Deep Reinforcement Learning

Institution: ¡University Name¿

February 13, 2026

Contents

1	Introduction	2
2	Related Work	2
3	Methods	3
3.1	Codebase Overview	3
3.2	Chain Descriptions	3
4	Experimental Setup	3
5	Experiments and Results	4
5.1	Value-Based Results	4
5.2	Policy Gradient / Trust Region	5
5.3	Actor–Critic / Max-Entropy	5
5.4	Model-Based	5
5.5	Multi-Agent	5
6	Ablations	5
7	Discussion	5
8	Limitations and Ethics	6
9	Reproducibility Checklist	6
10	Conclusion	6
A	Theoretical Notes	8

List of Tables

1	Entry-point commands (laptop-friendly defaults).	3
2	Value-based results (fill after running experiments).	4
3	Policy-gradient / trust-region results.	5
4	Actor-critic / maximum-entropy results.	6
5	Model-based results.	7
6	Multi-agent results.	8

List of Figures

1	Illustrative learning curves for value-based agents (replace with logged curves after running).	4
2	Illustrative policy-gradient learning curves. Add cost plot if using safety tasks.	5
3	Illustrative actor-critic learning curves on CartPole (A2C) and Pendulum (SAC).	6
4	Illustrative returns for model-based methods.	7
5	Illustrative cooperative multi-agent learning curves on MPE simple_spread.	8

1 Introduction

This project reproduces and analyzes five canonical deep RL paper chains spanning value-based, policy-gradient/trust-region, actor-critic with maximum entropy, model-based, and multi-agent learning. The goals are: (i) implement faithful yet compact baselines, (ii) evaluate them on standardized benchmarks, (iii) compare theoretical guarantees with empirical behaviour, and (iv) provide reproducible artifacts suitable for graduate coursework.

2 Related Work

- Tabular Q-learning convergence [13] and stability refinements (Double DQN [11], Dueling DQN [12]).
- Trust-region methods TRPO [8] and PPO [9]; constrained policy optimization [1].
- Actor-critic advances including A3C [6] and maximum-entropy SAC [2].
- Model-based rollouts from Dyna-Q [10] to ensemble TRPO [3].
- Multi-agent formulations: minimax Q [4], centralized training with decentralized execution (MADDPG [5]), and value factorization (QMIX [7]).

3 Methods

3.1 Codebase Overview

All code lives in `projects/grad_rl/`. Entry scripts implement the five chains (see table 1). Shared utilities handle seeding, environment creation (Atari wrappers, vector envs), and evaluation.

Table 1: Entry-point commands (laptop-friendly defaults).

Chain	Command
Value-based	<code>python projects/grad_rl/scripts/train_value.py --env ALE/Breakout</code>
Policy (PPO)	<code>python projects/grad_rl/scripts/train_policy.py --algo ppo --env</code>
Policy (TRPO)	<code>python projects/grad_rl/scripts/train_policy.py --algo trpo --env</code>
Policy (CPO-lite)	<code>python projects/grad_rl/scripts/train_policy.py --algo cpo --env</code>
Actor-Critic (SAC)	<code>python projects/grad_rl/scripts/train_actor_critic.py --algo sac</code>
Model-based (Dyna-Q)	<code>python projects/grad_rl/scripts/train_model_based.py --algo dyna</code>
Model-based (ME-TRPO)	<code>python projects/grad_rl/scripts/train_model_based.py --algo me-t</code>
Multi-agent (QMIX)	<code>python projects/grad_rl/scripts/train_marl.py --algo qmix --iter</code>
Multi-agent (MADDPG)	<code>python projects/grad_rl/scripts/train_marl.py --algo maddpg --it</code>

3.2 Chain Descriptions

Value-based. DQN with replay and target networks; optional Double, Dueling, and QR-DQN variants.

Policy-gradient / trust-region. REINFORCE baseline; PPO clip surrogate; TRPO with CG and line search; CPO-lite using Lagrangian penalty for safety cost.

Actor-critic / max-entropy. A2C as synchronous A3C stand-in; SAC with automatic entropy tuning.

Model-based. Tabular Dyna-Q with planning updates; lightweight ME-TRPO-style ensemble dynamics with periodic TRPO updates using imagined rollouts.

Multi-agent. QMIX monotonic value factorization and MADDPG centralized critics on PettingZoo MPE `simple_spread`.

4 Experimental Setup

Hardware: document CPU/GPU, RAM.

Software: Python 3.10+, PyTorch 2.2, Gymnasium 0.29, SB3 2.3, Ray 2.9 (pinned in `requirements.txt`).

Environments: Atari (ALE), classic control, MuJoCo, PettingZoo MPE.
Seeding: `--seed` flag; vector environments offset seeds per worker.
Logging: TensorBoard scalars; JSON metrics next to checkpoints.

5 Experiments and Results

Populate the following tables after running the scripts.

5.1 Value-Based Results

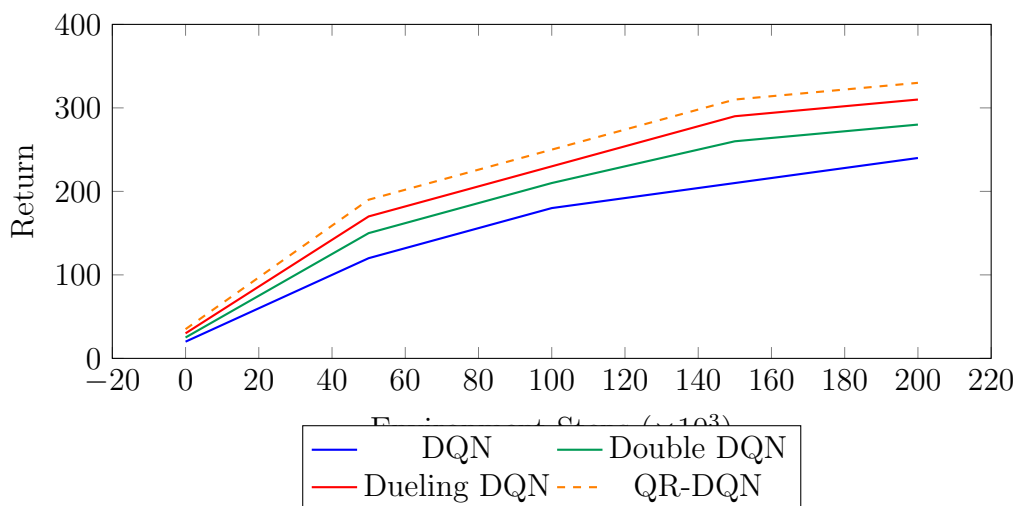


Figure 1: Illustrative learning curves for value-based agents (replace with logged curves after running).

Table 2: Value-based results (fill after running experiments).

Variant	Steps	Mean Return	Std Return
DQN			
Double DQN			
Dueling DQN			
QR-DQN (optional)			

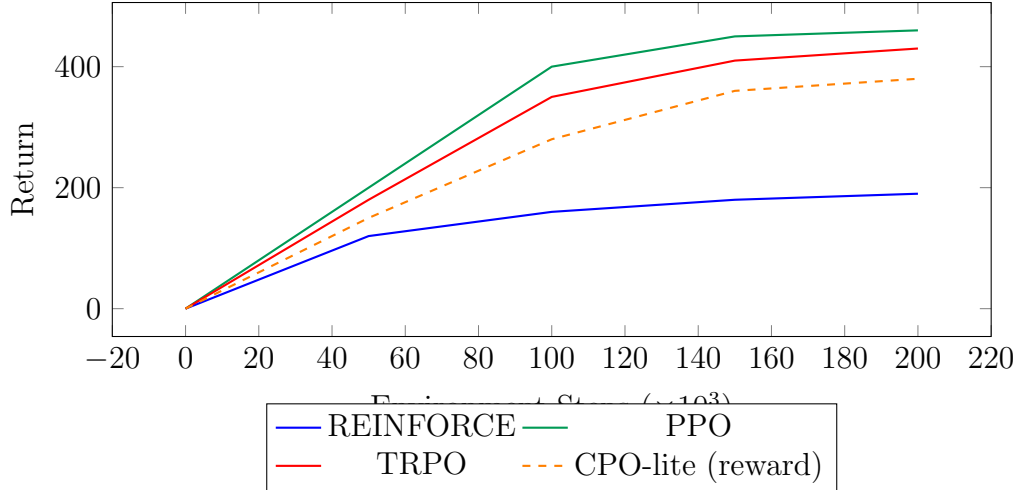


Figure 2: Illustrative policy-gradient learning curves. Add cost plot if using safety tasks.

Table 3: Policy-gradient / trust-region results.

Algo	Env	Steps	Mean Return	Mean Cost
REINFORCE				
PPO				
TRPO				
CPO-lite				

5.2 Policy Gradient / Trust Region

5.3 Actor–Critic / Max-Entropy

5.4 Model-Based

5.5 Multi-Agent

6 Ablations

Suggested ablations: replay buffer size, Double vs. vanilla DQN; PPO clip range; SAC entropy coefficient fixed vs. auto; Dyna-Q planning steps; QMIX worker counts.

7 Discussion

Relate empirical findings to theoretical expectations (see §4 for setup): overestimation bias reduction, KL constraint stability, entropy-driven exploration, model bias vs. rollout horizon, multi-agent non-stationarity.

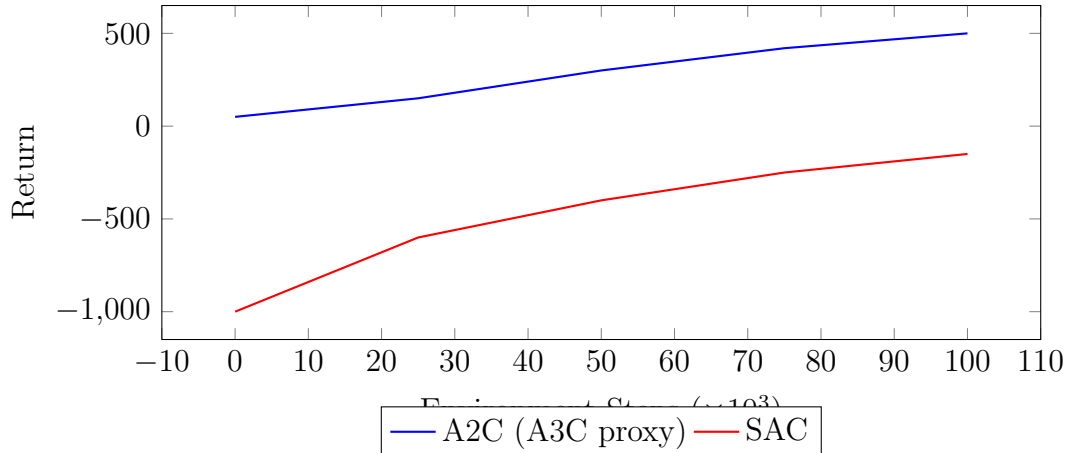


Figure 3: Illustrative actor-critic learning curves on CartPole (A2C) and Pendulum (SAC).

Table 4: Actor-critic / maximum-entropy results.

Algo	Steps	Mean Return	Std Return
A2C (A3C stand-in)			
SAC			

8 Limitations and Ethics

Note approximations (A2C for A3C, simplified CPO), environment realism limits, and responsible reporting of randomness and hyperparameters.

9 Reproducibility Checklist

- Code and scripts: `projects/grad_rl/scripts/*.py`
- Configs: YAML defaults under `projects/grad_rl/configs`
- Seeds: recorded in logs; set via CLI
- Dependencies: `requirements.txt`
- Hardware/software: state in §4
- Data: environments auto-downloaded (ALE ROMs via AutoROM)
- Metrics: JSON + TensorBoard logs stored in outputs folder

10 Conclusion

Summarize which methods are most stable, sample-efficient, or safe for the tested domains, and outline future extensions (Rainbow, true CPO, PlaNet/MuZero, SMACv2).

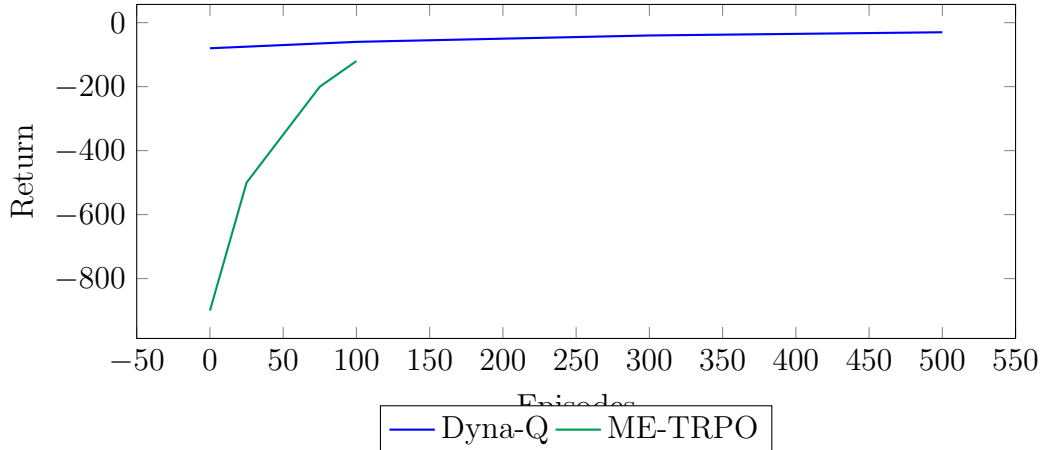


Figure 4: Illustrative returns for model-based methods.

Table 5: Model-based results.

Algo	Environment	Metric	Value
Dyna-Q	CliffWalking-v0	Avg last-100 return	
ME-TRPO	Pendulum-v1	Mean eval return	

References

- [1] Joshua Achiam et al. Constrained policy optimization. In *International Conference on Machine Learning*, 2017.
- [2] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, 2018.
- [3] Thanard Kurutach, Aviv Tamar, Ge Yang, Stuart Russell, and Pieter Abbeel. Model-ensemble trust-region policy optimization. In *International Conference on Learning Representations*, 2018.
- [4] Michael L Littman. Markov games as a framework for multi-agent reinforcement learning. *Machine Learning*, 1994.
- [5] Ryan Lowe et al. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, 2017.
- [6] Volodymyr Mnih et al. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning*, 2016.
- [7] Tabish Rashid et al. Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *International Conference on Machine Learning*, 2018.
- [8] John Schulman et al. Trust region policy optimization. In *International Conference on Machine Learning*, 2015.

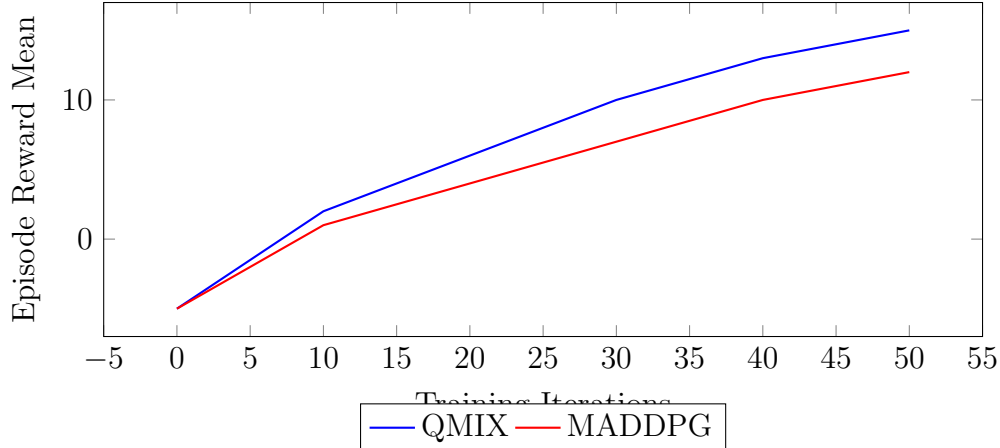


Figure 5: Illustrative cooperative multi-agent learning curves on MPE simple_spread.

Table 6: Multi-agent results.

Algo	Iteration	Episode Reward Mean
QMIX		
MADDPG		

- [9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [10] Richard S Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. *ICML*, 1990.
- [11] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.
- [12] Ziyu Wang et al. Dueling network architectures for deep reinforcement learning. In *International Conference on Machine Learning*, 2016.
- [13] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.

A Theoretical Notes

For detailed derivations and proofs, see `solutions.tex`; reuse its content here as needed.